Linux Processes & Threads

טרמינולוגיה 'של מתכנתים'

טרמינולוגיה של מתכנתים

מנקודת מבטו של המתכנת, הן ה- application programmer והן ה- system programmer, 'קיימים' שני מונחים שמייצגים ריצה/ביצוע של קוד:

*1. **תהליך** [= **process**] – תכנית בריצה.*

*2. **Thread** – 'קטע' {פונקציה} של תכנית. ש'יכול' לרוץ במקביל ל- thread-ים אחרים של התכנית. היינו, לפונקציות אחרות.*

טרמינולוגיה של מתכנתים

- ברוב המכריע של הספרות הטכנית המונח '**תהליך**' מוגדר כ**תוכנית בריצה**.
- כמו כן המונח **תהליכון** [= **thread**] מוגדר כרצף ביצוע קטן ובלתי תלוי בתוך תהליך (Process)

- ההגדרות האלו הן 'בעייתיות' מכיוון שהן לא מתארות למעשה את המהות של המונחים.
- ולכן במשך נביא הגדרות אחרות

```
#include <stdio.h>          /* standard I/O routines              */

/* Global initialized data/variables
'resides' in the data section of the executable file */
int Gx = 3;
int Gy = 17; /* in the data section */

/* Global uninitialized data/variables.
'resides' in the bss section of the executable file */
int Gz ;
int arr[1000];

int main(int argc, char* argv[])
{

/* local data/variable
  'resides on the stack. Only at run time. */
   int       t;

   f1(t) ;
   f2(t) ;

}
```


- *Sequential program*
- *or: Single process*
- *or: Single threaded process/program*

```
#include <stdio.h>          /* standard I/O routines                */
#include <pthread.h>     /* pthread functions and data structures */

int Gx = 3;  /+ in the data section */
int Gy = 17; /+ in the data section */

int Gz ;   /* in the bss section */
int arr[1000];  /* in the bss section */

int main(int argc, char* argv[])
{

   pthread_t  thread_id1, thread_id2;          /* thread's ID (just an integer)         */
   int       t        = 11;   /* data passed to the new thread         */

   /* create a new thread that will execute. T1*/
   pthread_create(&thread_id1, NULL, f1, (void*)t);

   /* create a second new thread that will execute. T2*/
   pthread_create(&thread_id2, NULL, f2, (void*)t);

   pthread_join( thread_id1, NULL );
   pthread_join( thread_id2, NULL );
}
```

- Concurrent program
- or: Multi threaded process/program

טרמינולוגיה של ה- Linux kernel

טרמינולוגיה של ה- kernel: תהליך [= process]

- ברוב המכריע של הספרות הטכנית המונח '**תהליך**' מוגדר כ**תוכנית בריצה**.
- ההגדרה הזו בעייתית מכיוון שהיא לא מתארת למעשה את המהות של המונח.
- ולכן נביא הגדרה אחרת:

תהליך הוא אובייקט/ישות [= entity] שנוצר על ידי מערכת ההפעלה ו'בהמשך' מנוהל על ידי מערכת ההפעלה.

תהליך הוא אובייקט/ישות [= entity] שנוצר על ידי מערכת ההפעלה ו'בהמשך' מנוהל על ידי מערכת ההפעלה.

הסברים/הערות להגדרה:

א. תהליך נוצר על ידי מ"ה בעקבות בקשה [= קריאת מערכת] של תהליך {שכבר <u>קיים</u>}.

- לדוגמא: בעקבות בקשה של תהליך ה- bash

- דוגמא נוספת: בעקבות בקשה של תהליך server שמקבל פניה {בקשת שירות} מתהליך client.

- דוגמא נוספת: בעקבות בקשה של תהליך ה- chrome להרצת adobe acrobat reader. לצורך הצגת התוכן של קובץ בפורמט pdf.

עם יצירתו

ב. אובייקט/ישות = מיוצג על ידי מערכת ההפעלה 'באמצעות' רשומה [= struct]

הסברים/הערות להגדרה – המשך:

ג. מבחינת מערכת ההפעלה 'מדובר' באובייקט שמשויך לו קוד [= הוראות מכונה] ש'צריך' להריץ/לבצע

ד. 'צריך' להריץ = על ידי ה- CPU

היינו, תהליך הוא אובייקט שעל מ"ה, בתור מנהלת משאבי החומרה של המחשב בו היא רצה, להקצות לו זמן CPU

ה. לצורך הרצה ע"י ה- CPU הקוד והדאטה חייבים 'להימצא' ב- MM. לא ב- NVM.

ולכן, לצורך הרצת הקוד המשויך לתהליך, מ"ה חייבת להקצות לו <u>מקום פנוי ב- MM</u>

המקום הזה נקרא מרחב הכתובות של התהליך [process address space]

ו. בנוסף להקצאת רשומת תהליך, ולהקצאת מקום פנוי ב- MM בזמן יצירת תהליך חדש מערכת ההפעלה 'מקצה' עבורו משאבים נוספים.
משאבים אלו הכרחיים, בהמשך, לצורך הרצת הקוד המשויך לתהליך.
לדוגמא: שלשה FDs: stdin, stdout, stderr

הערות לגבי מרחב הכתובות של תהליך

הערות לגבי מרחב הכתובות של תהליך:

- מערכת ההפעלה היא זו שמקצה את המקום ב- MM, מכיוון ש'בתוקף' תפקידה כמנהלת משאבי החומרה היא מנהלת גם את ה- MM.
- אל המקום הזה ב- MM מ"ה 'מעתיקה' את הוראות המכונה השמורות בקובץ ההרצה [= executable] ואת המשתנים הגלובליים המאותחלים.

בנוסף, מערכת ההפעלה מקצה מספיק מקום במרחב הכתובות של התהליך עבור:

- משתנים גלובליים לא מאותחלים
- אזור שינוהל כמחסנית [= stack]
  - ישמש לצורך 'העברת' פרמטרים לפונקציות וכן לצורך הקצאת מקום עבור משתנים לוקאליים
- אזור שינוהל כערמה [= heap]
  - ישמש עבור הקצאה דינמית [= בזמן ריצה] של זיכרון.
  - למשל הקצאה 'בעקבות' קריאה ל- malloc()

יצירת תהליך: סיכום

- ה- kernel יוצר תהליך חדש בעקבות בקשה {= קריאת מערכת} 'שמגיעה' ל- kernel מתהליך קיים.
- קריאת המערכת המבקשת ליצור תהליך חדש בלינוקס: fork()
  - נתאר בהרחבה בהמשך את הסמנטיקה ואת ערך החזרה של fork()
- הקריאה, כמו כל קריאות המערכת, 'גורמת' ל- kernel לבצע סדרת צעדים 'מתאימה' [= כזו שביצועה יספק/יממש 'את מה שמבקש' התהליך שביצע את קריאת המערכת], ובכלל זה:

1. הקצאת רשומת תהליך
   - 'מייצגת' את התהליך = 'שומרת'/'מכילה' את ה- meta data על התהליך
   - בלינוקס שמה של רשומה זו הוא task_struct

הערה: בספרות שעוסקת במערכות הפעלה ללא 'התייחסות' ספציפית ללינוקס נהוג להשתמש במונח 'רשומת PCB', או בקיצור PCB [= process Control Block]

2. הקצאת 'מזהה תהליך' [= process id]
   - מספר שלם אי שלילי
   - בלינוקס הערך 0 'שמור' עבור תהליך 'מיוחד' שנקרא ה- idle process, והערך 1 'שמור' עבור תהליך ה- init {שהוא התהליך שנוצר עם תחילת ריצת הקוד של ה- kernel}

3. הקצאת מרחב כתובות

יצירת תהליך: סיכום (המשך)

4. הקצאת שלשה FDs
5. הקצאת משאבים נוספים
6. צעדים נוספים

כל אוסף הצעדים שמבצע ה- kernel 'בעקבות קבלת' בקשה לחצירת תהליך חדש [= בעקבות הקריאה ()fork] 'נועד' ליצור את התנאים ההכרחיים לריצת הקוד המשויך לתהליך !!!

'ניהול' ריצת תהליך

כפי שהסברנו קודם בתיאור, מבחינת מערכת ההפעלה 'מדובר' באובייקט שמשויך לו קוד [= הוראות מכונה] ש'צריך' להריץ/לבצע על ידי ה- CPU
היינו, תהליך הוא אובייקט שעל מ"ה, בתור מנהלת משאבי החומרה של המחשב בו היא רצה, להקצות לו זמן CPU

- לכן, פעם שמערכת ההפעלה סיימה ליצור תהליך חדש Pchild, והתבצעה חזרה 'מוצלחת' מהקריאה ל- fork(), היא תנהל את ריצת התהליך.
- עד מתי? עד סיום ריצתו [= 'מחיקתו' מהמערכת]
  - סיום 'עצמי' תקין.
    - לדוגמא, בעקבות קריאה של התהליך Pchild עצמו ל- exit()
  - סיום 'כפוי' מתהליך אחר [= סיום 'לא עצמי']
    - לדוגמא, בעקבות קריאה ל- kill() שמבצע Pparent [= התהליך שיצר את Pchild. היינו, התהליך שקרא ל- fork()]
  - סיום 'כפוי' בעקבות ביצוע שגיאה תוך כדי ריצה.
    - לדוגמא, בעקבות ניסיון חלוקה ב- 0.
  - דוגמא אחרת: בעקבות ניסיון גישה ל- MM לכתובת שאינה 'שייכת' למרחב הכתובות שהוקצה ל- Pchild

'ניהול' ריצת תהליך – המשך

מה הכוונה ב'ניהול ריצת התהליך'?
- ניהול 'מעברי מצבים' של התהליך

טרמינולוגיה של ה- kernel: תהליכון [= thread]

- כפי שראינו, מבחינת המתכנת תהליכון [= thread] הוא 'קטע קוד' {פונקציה} של תכנית.
- ש'יכול' לרוץ במקביל ל- thread-ים אחרים של התכנית.
- היינו, לפונקציות אחרות של התכנית.

- מבחינת ה- kernel, ה- thread הוא אובייקט שיש:
  1. - ליצור אותו
     - יש הבדל בין
       - ה- main thread
       - נוצר 'אוטומטית' [= ללא בקשה מפורשת מה- kernel] עם יצירת התהליך,
       - ואשר משויך לו הקוד של הפונקציה main,
       - לבין 'כל שאר' ה- Threads אשר נוצרים בעקבות קריאת מערכת ל- kernel
       - pthread_create()
  2. - לנהל אותו

'ניהול' ריצת thread

מה הכוונה ב'ניהול ריצת התהליך'?
- הקצאת זמן CPU {מעת לעת}
- 'מתן שירותים' ל- thread בעקבות קריאות מערכת שהוא מבצע
- כמו למשל קריאה ל- open()
- או, קריאה ל- fopen() {שהמימוש שלה 'כולל', כפי שראינו בהרצאות הקודמות, קריאה ל- open() }

Linux Tasks

יצירת קובץ executable

- המתכנת כותב את התכנית ושומר בקובץ example.c
- מקמפל: gcc example.c -o example
- נוצר קובץ executable ששמו example

- מקליד {במסך ה- Terminal של ה- bash}: ./example
- בקשה להריץ את התכנית השמורה בקובץ example. ב- current directory.

יצירת תהליך בן

- תהליך ה- bash קורא מהקלט את הפקודה [= את ה- string] example
- מבצע parsing של הפקודה
- מזהה שמדובר בפקודה שהוא {ה- bash} לא יודע 'לבצע בעצמו'
- ולכן: מבצע קריאה ל- fork() [= קריאת מערכת]
  - מבקש ממ"ה שירות של יצירת תהליך חדש

- כפי שראינו בהרצאות הקודמות, מנקודת מבטו של תהליך ה- bash הוא מבקש יצירת תהליך חדש תהליך בן [child process]
- מנקודת מבטה של מ"ה לינוקס: היא מתבקשת ליצור אובייקט חדש מסוג task

להלן 'שלד' של הקוד של ה- bash:

A trivial bash

```
while(1) {
        type_prompt();
        read_command(command);   /* input from terminal */

        parse(command) ;
         /* 'break' the command {= string} into 'tokens'.
            the 1st token is considered to be the command name
            whereas the other tokens are considered as arguments or flags to the command */

        /* now, check if the command name is one that bash can perform.
                for example 'cd' or 'history', etc.
           if yes – carry out the appropriate operations
           if not – create a new process. a child process.
                - the child process is inherits the resources allocated to the parent process,
                        that is to the bash process.
                - including all memory areas [text {code} area, data area, bss area, etc.] allocated to bash
                - open file descriptors
                - other resources
        */
```

A trivial bash (cont.)

```
pid = fork();  /* request to create a child process*/

if(pid < 0) {
        printf("Unable to fork");
        continue;
}

if (pid != 0) {
        /* parent [= bash]  code... */
}
```

A trivial bash (cont.)

```
        else {
                ***execvp(command, params,);  /* child  code ... */***
        }
        /* request 'to return' the allocated resources.
           that is:
              'return' {'disconnect'} all allocated memory areas
              'return' {close} all open file descriptors
              etc.
           and allocate 'new' resources
              text memory area into which to copy the code [machine instructions] in the file 'example'
              data area – for Gx and Gy
              bss area – for arr, and Gz
        …..
        ….


} /* end of while loop */
```

המימוש של fork()

- מימוש ה- fork() נעשה ע"י מ"ה

- מקצה task_struct חדש
- רושמת בשדה 'pid' ייחודי [כזה שלא מוקצה לאף אחד מה- task-ים המנוהלים כרגע ע"י לינוקס]
- רושמת בשדה 'ppid' את ה- pid של תהליך ה- bash
- ...
- מקצה רשומת mm_struct חדשה
- שומרת מצביע לרשומה זו בשדה mm {של רשומת task_struct}
- ברשומה mm_struct שומרת ערכים השמורים ברשומת ה- mm_struct המשויכת ל- task_struct שמייצגת את ה- bash

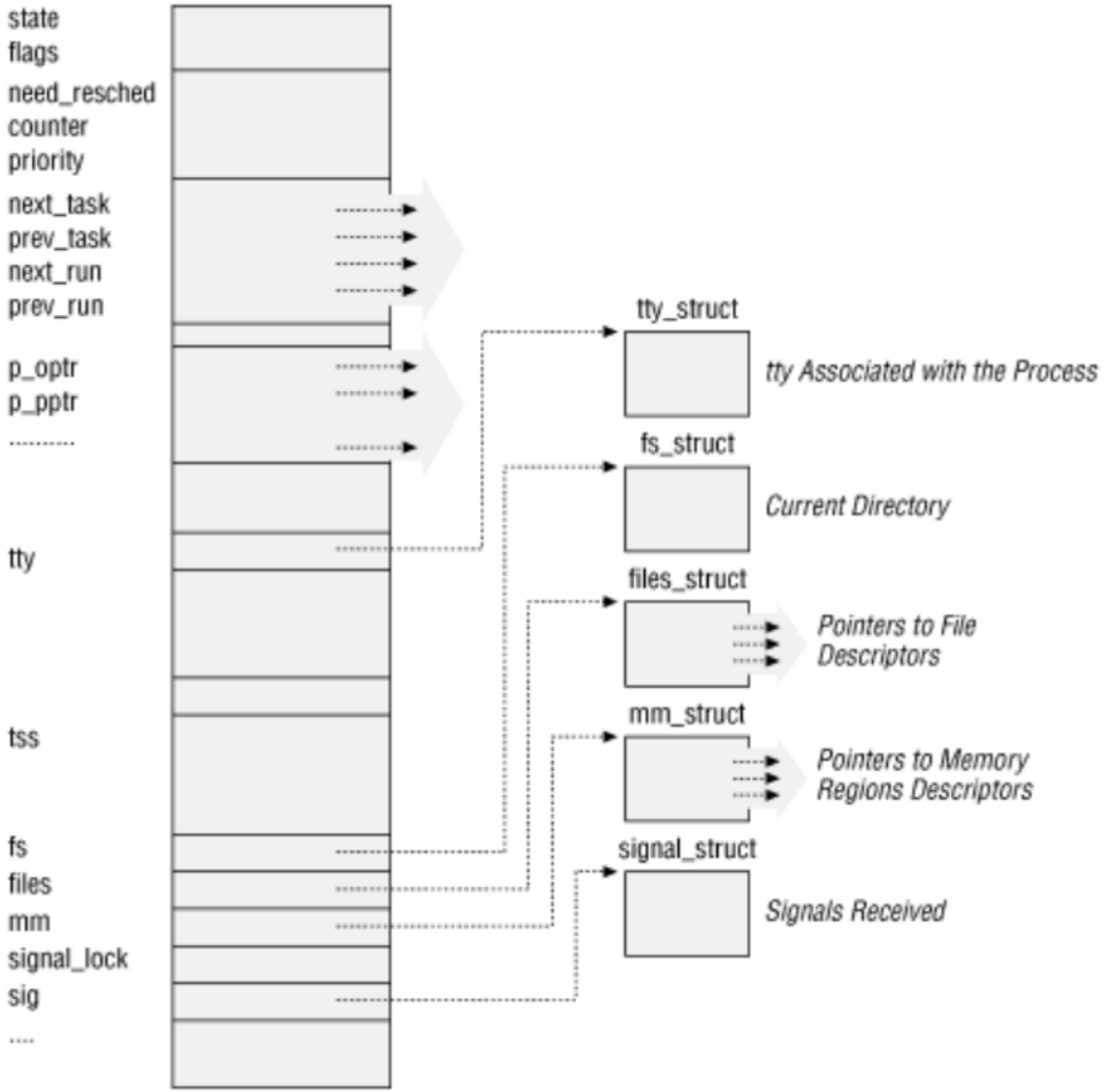
Example: Process Representation in Linux

Represented by the C structure ***task_struct***

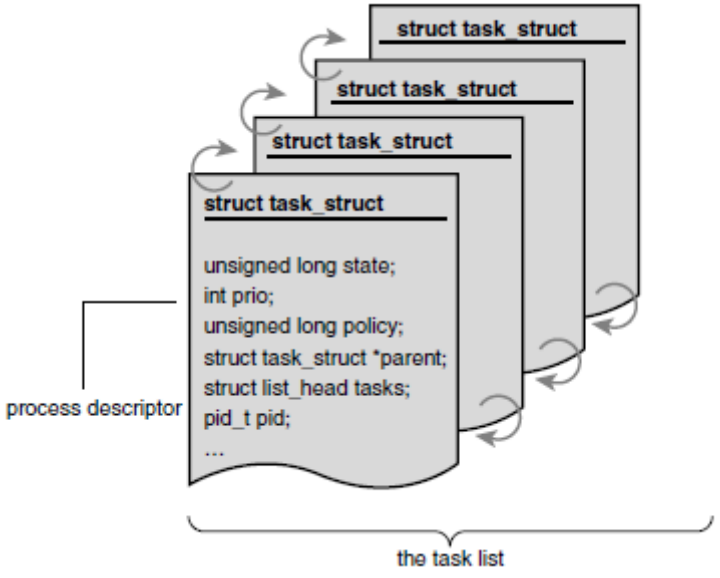
```
pid t_pid;                    /* process identifier  */
long state;                       /* state of the process      */
unsigned int time_slice           /* scheduling information    */
struct task_struct *parent; /* this process's parent   */
struct list_head children;  /* this process's children       */
struct files_struct *files; /* list of open files     */
struct mm_struct *mm;     /* address space of this process */
```

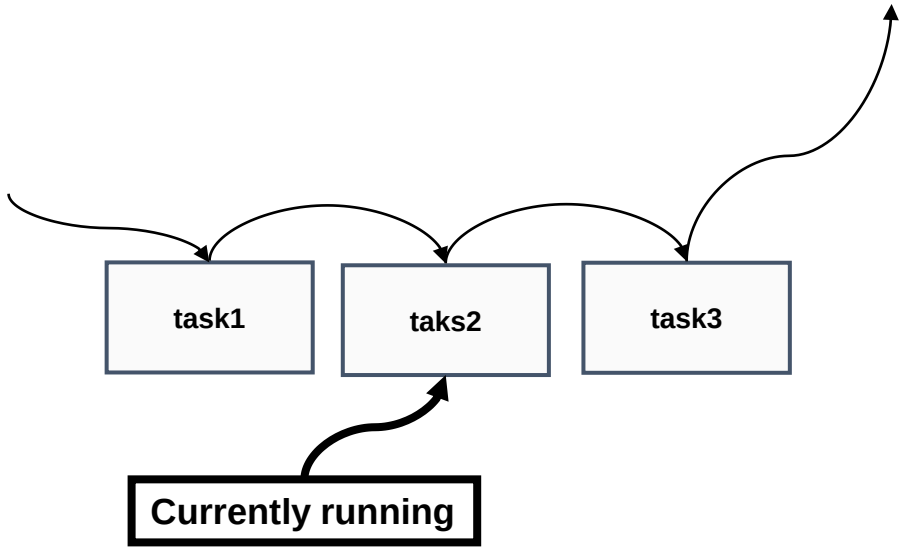
The linux process descriptor: the task_struct

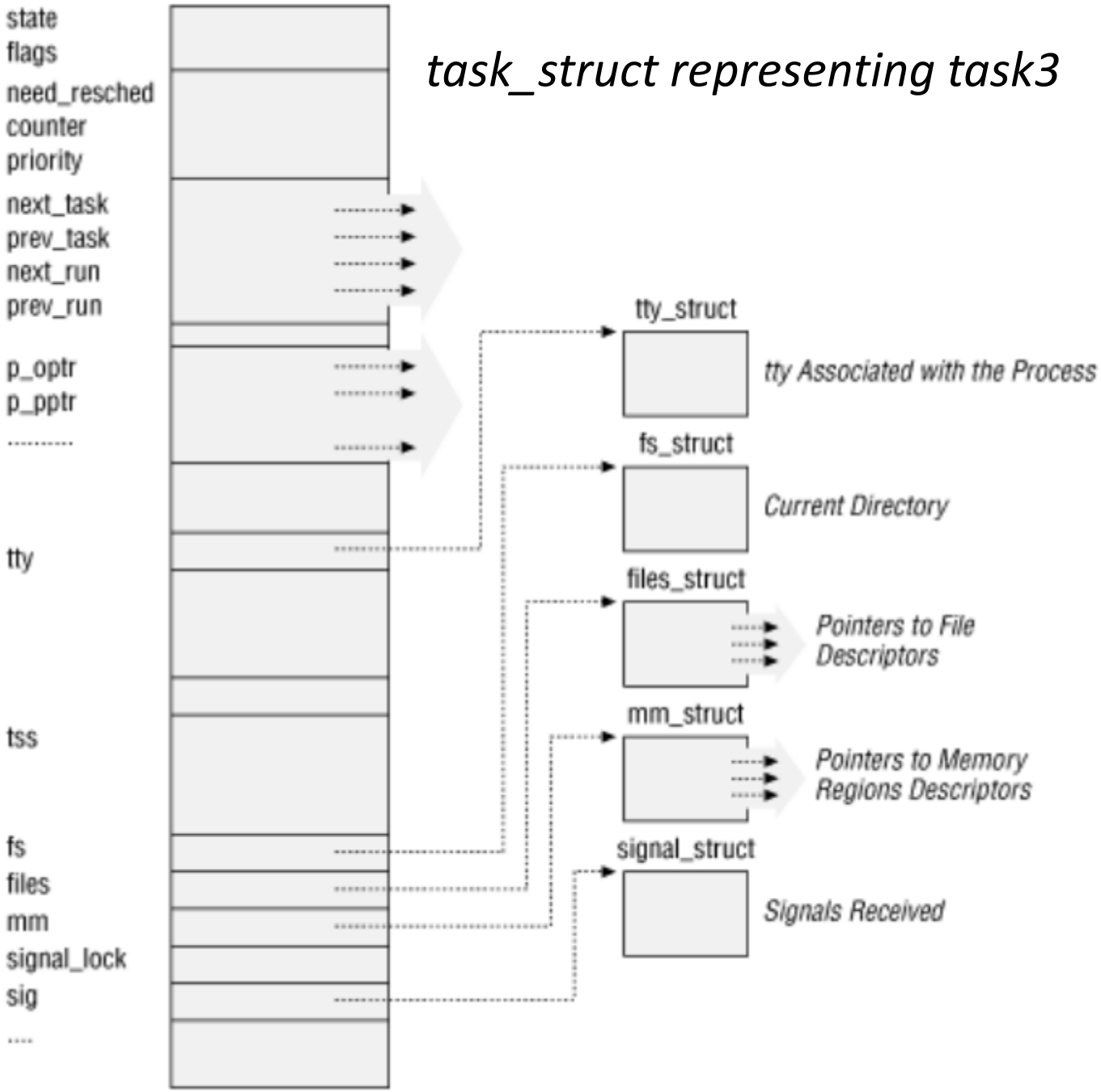
The five data structures on the right side of the figure refer to specific resources owned by the process.



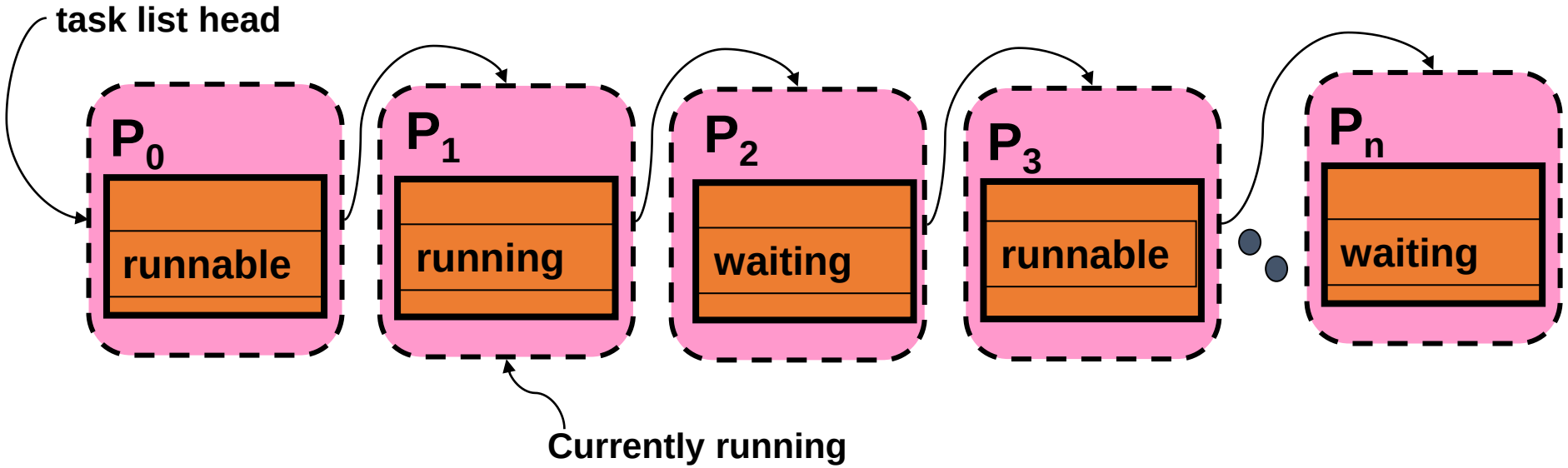
The Linux task list [= process list]




task1
taks2
task3
Currently running


task_struct representing task3
state
flags
need_resched
counter
priority
next_task
prev_task
next_run
prev_run
p_optr
p_pptr
.....
tty
tss
fs
files
mm
signal_lock
sig
....
tty_struct
tty Associated with the Process
fs_struct
Current Directory
files_struct
Pointers to File Descriptors
mm_struct
Pointers to Memory Regions Descriptors
signal_struct
Signals Received

The task list



המימוש של exec()

מימוש ה- exec() נעשה ע"י מ"ה

רושמת NULL בכל השדות של רשומת ה- mm_struct אשר מצביעים ל- memory areas שמוקצים ל- bash
מקצה memory araes חדשים
שומרת מתביעים אליהם ברשומתה- mm_struct של תהליך הבן
מעתיקה את הוראת המכונה מ- example ל- text area
מאתחלת את המקומות ב- data section המוקצים ל- Gx ול- Gy {ל- 3 ול- 17, בהתאמה}
וכו'

task life cycle & state transitions

- כל זמן 'חייו' (= מרגע יצירתו {ויצירת רשומת ה- task_struct שלו} של T ועד סיום ריצתו {ומחיקת רשומת ה- task_struct שלו}) במערכת - משתנה המצב של T:
  - מצב NEW בו נמצא T מרגע יצירת רשומת ה- task_struct ועד סיום כל הצעדים הדרושים ליצירתו
  - מצב RUNNIG בו נמצא T כאשר המעבד מוקצה לו, ומבצע את הוראות מכונה השמורות ב- text area שלו
  - מצב TERMINATE בו נמצא T מרגע תחילת הטיפול בסיום ריצתו ועד רגע מחיקת רשומת ה- task_struct המקצת לו
  - ומצבים נוספים

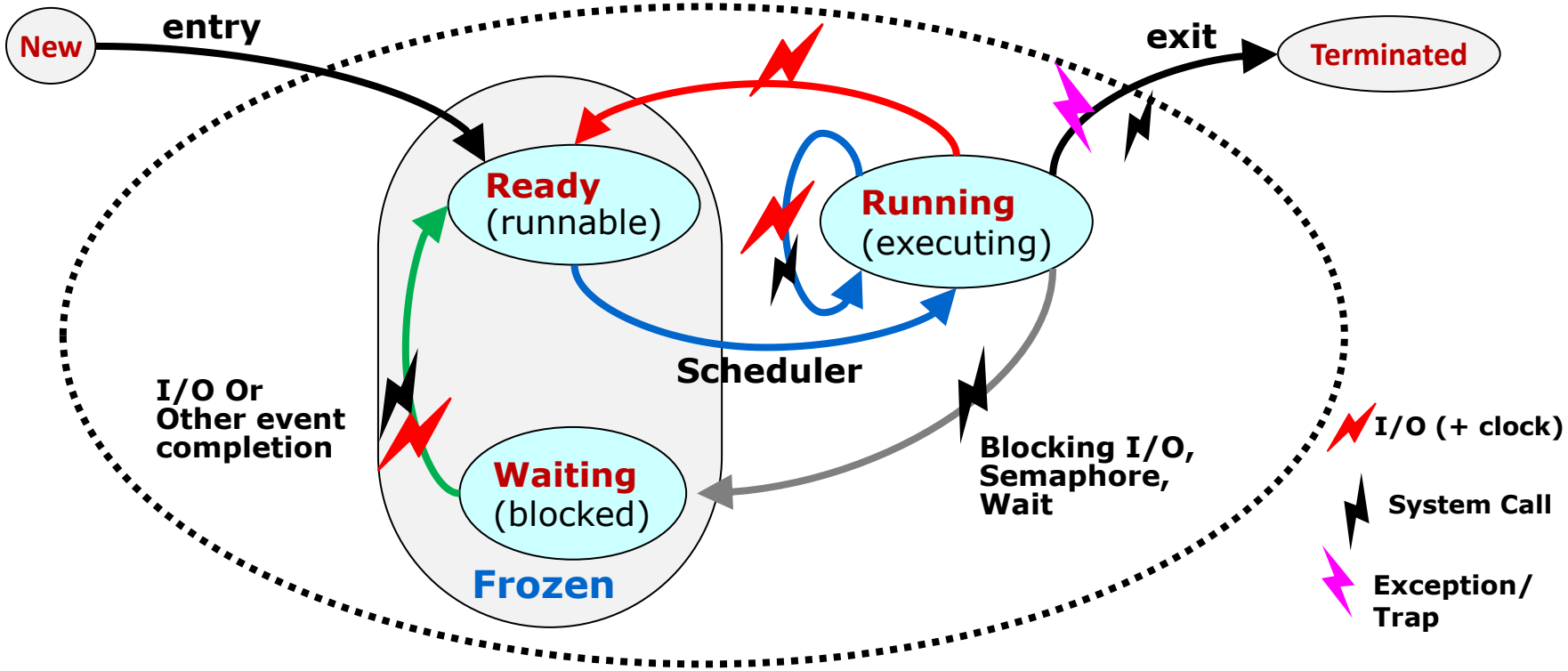
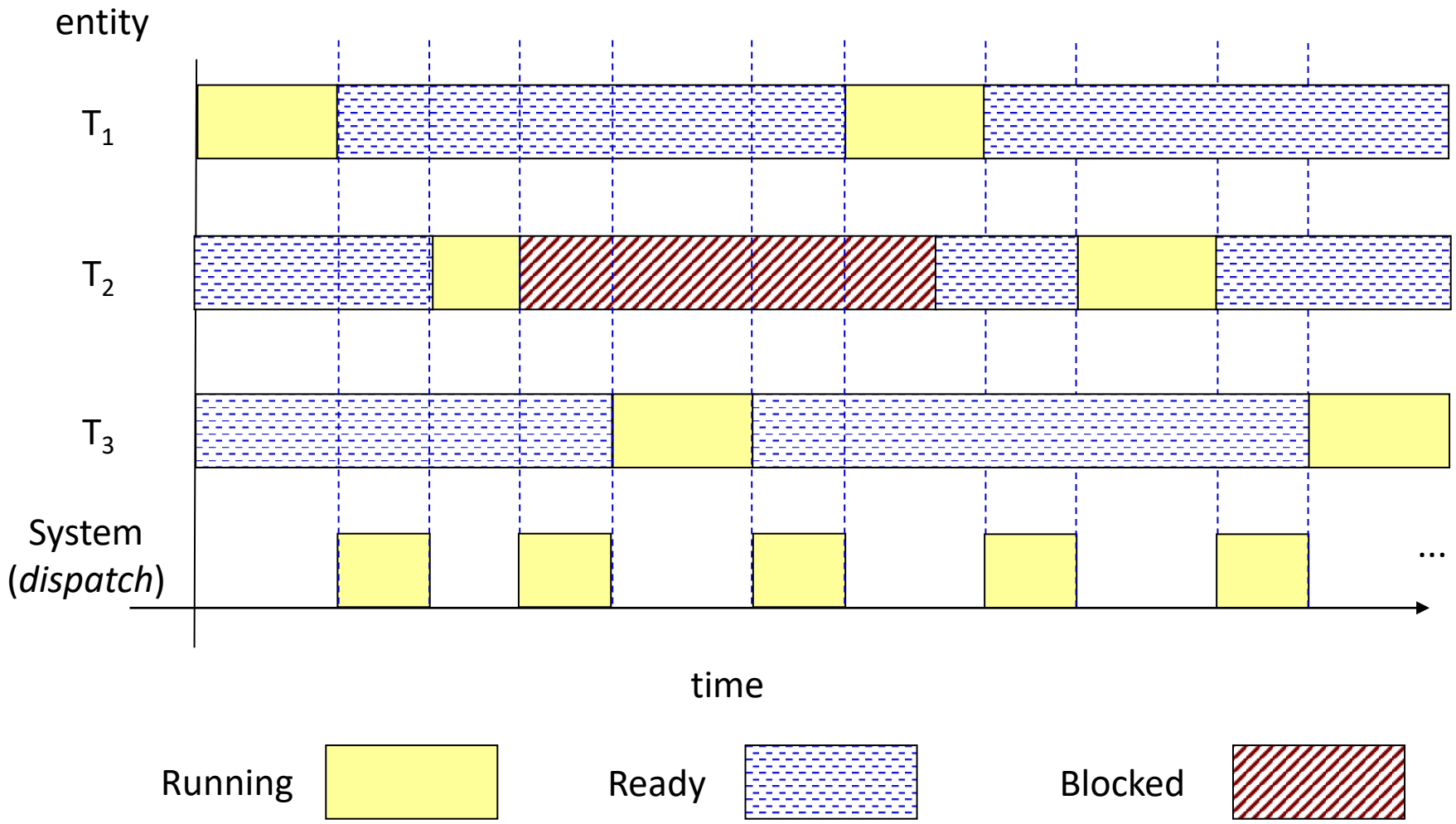
- באופן כללי:
- מספר המצבים בהם תהליך יכול להימצא תוך כדי חייו, והשמות שלהם – משתנה בין מערכות הפעלה שונות.
- בספרות העוסקת במ"ה מתוארים באופן טיפוסי 5 מצבים והם:
  - New, Ready, Running, Waiting, Terminate

- בהמשך נרחיב על מצבים אלו, ונתאר מה מציין כל מצב (= מתי תהליך נמצא בכל אחד ממצבים אלו) וכן את הסיבות למעברי המצבים.

States & transitions

As far as the OS is concerned - as a **process** progresses, it changes **state**

- **New**: The new process is being created
  - **Ready**: The process is ready to be assigned to a processor
  - **Running**: Instructions are being executed
  - **Waiting**: The process is waiting for some event to occur
- **Terminated**: The process has finished execution



Movement among States: 3 Tasks Example